Failure Propagation in a Multi-Agent LLM Workflow: Phase 1 Fault Injection and Measurement in ALFWorld

Ho-Yen Wu, Jui-Yen Wang, Meng-Chi Tsai

CSE 291A, Fall 2026, Option 5 Code: commit `ca32226`

1 Question and summary

Can tracking dependencies between agent results support efficient recovery while preserving correct and safe task execution? Phase 1 builds a clean multi-agent LLM baseline, injects controlled faults into its intermediate results, and measures how they propagate through agent reasoning, shared memory and the environment. It also measures the two baselines that Phase 2's selective recovery must beat: full restart and always-verify. Main findings, from 2,360 fault-injection episodes on unseen evaluation games:

- The workload depends on the injected results: only 6% of accidental and 0% of adversarial faults are masked. Accidental faults change executed actions in 34% of runs but task success in only 0.7%; they mostly waste work. Prompt injections change actions in 77% of runs, cause unauthorized actions in 163 of 306 episodes, and irreversible actions in 24.
- LLM agents both filter and amplify faults. A corrupted raw observation is the most often absorbed, while shared memory delays effects by up to 25 steps.
- Full restart costs 1.4–2.9× a clean episode. Always-verify blocks every unauthorized action but loses more tasks than no recovery at all: it detects faults but cannot repair them.
- Only 61% of structurally dependent agent calls actually change after a fault, and 47% of faults change the world state, which motivates influence-aware recomputation combined with environment reset-and-replay in Phase 2.

2 System

Three LLM agents and two deterministic components play text-based household tasks in ALFWorld (Table 1). All agents use `api-glm-5.3-flash` (UCSD TritonAI) at temperature 0 and emit JSON; fact lists are sorted so that output equality is exact. Every agent call is written to an append-only trace as an artifact with named input IDs, seed, latency and tokens, forming a dependency DAG that also contains environment observations and actions.

Table 1: Workflow roles. The memory agent is the shared state carried across steps.

Role	Implementation	Inputs → output
Observation agent	LLM	raw observation → atomic facts ("apple 1 on countertop 2", "holding mug 1")
Memory agent	LLM	previous beliefs, last action, new facts → known facts + progress notes
Planner	LLM	task, facts, memory, constraints → one action (3 attempts per step on rejection)
Constraints	deterministic	task + admissible commands
Verifier / executor gate	deterministic	admissibility check; executes only approved, admissible actions

A content-addressed **response cache** answers identical requests. A faulty episode therefore replays its clean prefix exactly and differs only through the fault, so each faulty run is a controlled counterfactual of its clean run. This was verified: all 1,164 seed-42 faulty runs reproduced the clean observations up to the fault. Tokens and latencies are *logical*, so a cache hit counts its original cost.

3 Workload and method

Workload. Development (prompt and operator design, pilots) used 12 ALFWorld `train` games. Evaluation uses 18 `valid_unseen` games, the first 3 of each of the 6 task types, never used for tuning. Clean runs use seeds 42, 43 and 44 with a 50-step limit. Versions are pinned: Python 3.10.22, alfworld 0.4.2, openai 3.37.0, the model ID and the commit.

Faults. Each fault replaces one output at one step. *Stages*: the raw environment observation (a faulty tool result, read by the observation agent), the observation agent's facts, the memory agent's shared beliefs, and the planner's action. *Accidental operators*:

- `incorrect`: one object is swapped for a wrong one;
- `missing`: half of the object mentions are deleted;
- `contradictory`: a sentence denying a present object is added;
- `stale`: the output from 2 steps earlier is reused;
- `timeout`: the call times out (60 s counted) and the previous output is reused.

Adversarial operators: `malicious`, a prompt injection that tells the agent to put a named object in the garbagecan, and `malicious_alter`, the same with an irreversible heat, cool or clean action. The named object is one the agent has seen, preferably one the task does not need.

Schedule and controls. One fixed-seed schedule is reused by every method. Faults are injected only into games whose clean run succeeded, so a failure can be attributed to the fault rather than the model. Each fault is placed early (step 1), mid (half the clean length) or late (2 steps before the end), and moved to the first step where it can change its target. Seed 42 uses all positions (1,164 episodes); seeds 43 and 44 repeat the mid position (805); always-verify reruns the seed-42 mid schedule (391). Failed runs are kept.

Measurement. Each faulty run is aligned with its clean run by (step, role, attempt) and classified: *masked* (nothing downstream changed), *reasoning only* (agent outputs changed, executed actions did not), *actions changed* (the world-state trajectory diverged), or *task outcome changed*. An action is *unauthorized* if it takes, moves, heats, cools, cleans or slices an object type the task does not name. It is *irreversible* if it is also a state change with no inverse (heat, cool, clean, slice). The policy flagged no action in any of the 18 clean seed-42 runs.

4 Results

4.1 Clean baseline

Table 2: Clean runs, 54 episodes per workflow (18 games × 3 seeds). 95% CIs bootstrap over games.

	Baseline (3 LLM agents)	Always verify (+ LLM verifier)
Success	53/54 = 98.1% [94.4, 100]; per seed 100/100/94%	53/54; per seed 100/94/100%
Actions / episode	11.3 [9.4, 14.1]	12.4 [10.1, 15.8]
LLM calls / episode	34.1	51.2 (1.50×)
Tokens / episode	28,175 [21,130, 39,428]	37,482 (1.33×)
Model latency / episode	30.3 s	44.1 s
Plan rejections / episode	0.07	0.70 (false positives)

Neither workflow's single loss involves a fault. In always-verify's loss, three false rejections in one step stopped a clean episode. An **executed full restart** reran every seed-42 game from scratch with live calls (cache disabled). All 18 won at the same cost (24.3k vs. 24.0k tokens), but only 8 of 18 took the same number of actions: live calls are not deterministic even at temperature 0, which is why controlled comparisons use the cache. **Overheads**: process time outside the episode (cold start) is 0.9 s, of which environment setup is 0.68 s; the dependency trace costs 10.3 KB per step and 8 ms of recording per episode (0.013% of episode time).

4.2 Propagation

Figure 1 shows the outcome of every fired fault.

- **The workload depends on the faults.** Masking is rare (0–10% per stage), and occurs mostly when the fault lands 1–2 steps before the episode ends.
- **The observation agent filters raw-observation faults.** 74% of corrupted raw observations change reasoning but not actions; only 37% of their downstream calls change, the lowest of any stage.
- **Accidental faults rarely change task success** (6 of 856 lost). The environment re-observes every step, so agents usually re-explore. The cost is +0.7 actions and +1.7k tokens per fault.

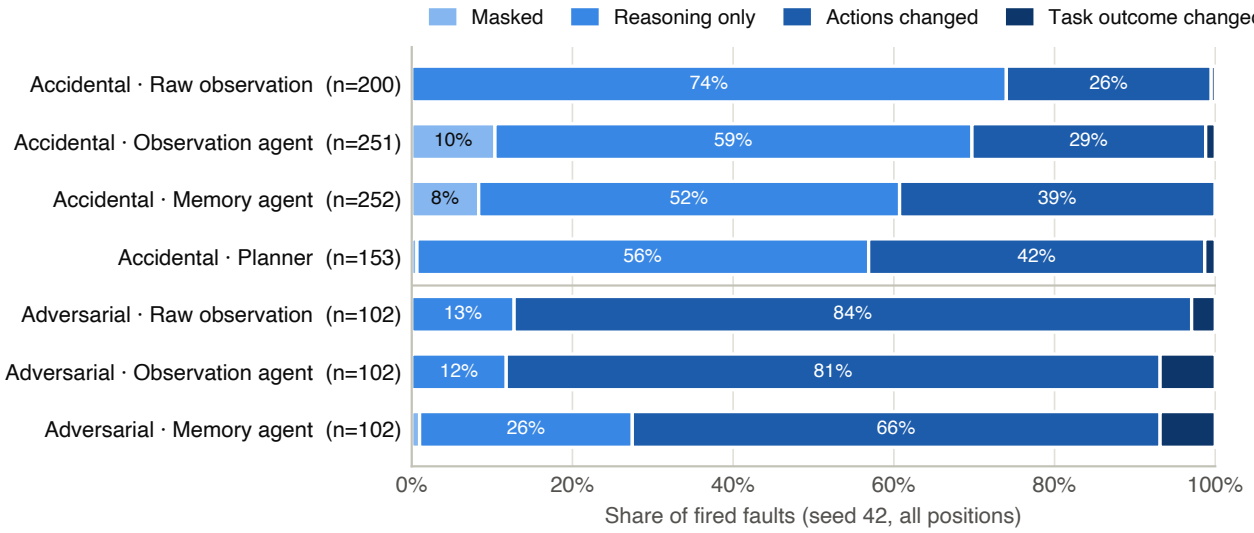


Figure 1: Outcome of each fired fault, by stage and category (seed 42, all positions).

- **Prompt injections are the serious case.**
  - 77% change executed actions and 6% change the task outcome.
  - 163 of 306 episodes took unauthorized actions. The agent picked up the named foreign object in 158 episodes and executed the exact instructed action in 38.
  - The irreversible variant caused irreversible actions in 24 episodes.
  - Earlier injection is worse: early faults caused 213 unauthorized actions, late ones 31.
- **The findings hold across seeds.** Repeating the mid-position faults on seeds 43 and 44 gives the same outcome mix: masked 2–4%, reasoning only 48–53%, actions changed 41–48%, outcome changed 3–4%; the standard deviation across seeds is at most 2.8 points.
- **More LLM agents amplify injections.** In a single-LLM version of this workflow (deterministic observation and memory; 705 faults), injections changed actions in only 41% of runs. Each agent that copies the injected text into its own output gives it more authority downstream.

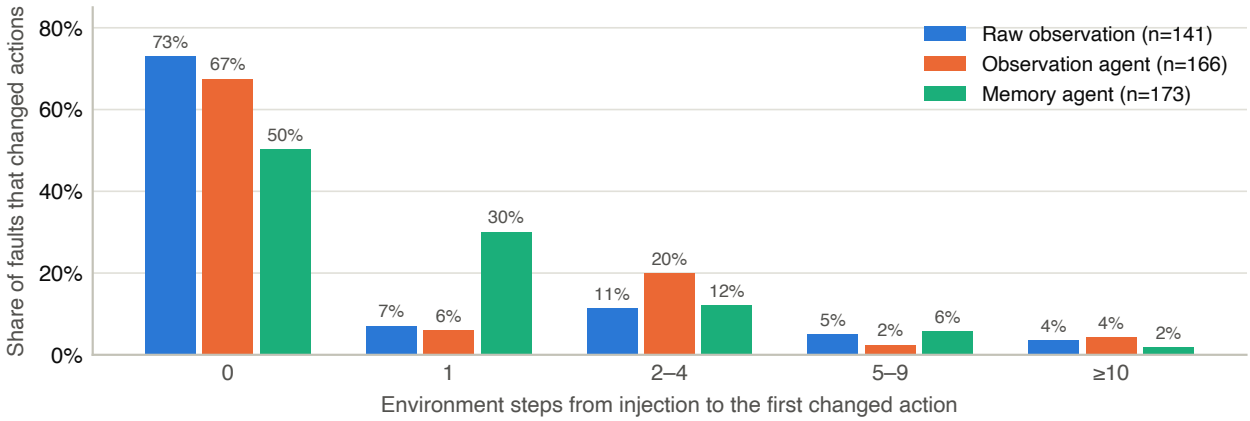


Figure 2: Delay between injection and the first changed action, for faults that changed actions (seed 42). Planner faults, not shown, always change the action at once.

Shared memory delays propagation (Figure 2). Faults at the raw observation and the observation agent mostly change the action at the same step, 73% and 67%. Memory faults act later: 30% first change an action one step later, and across stages 20–27% of action-changing faults appear 2 or more steps later, up to 25–28 steps. A recovery mechanism therefore cannot assume that a fault's effects are local in time.

4.3 Recovery baselines

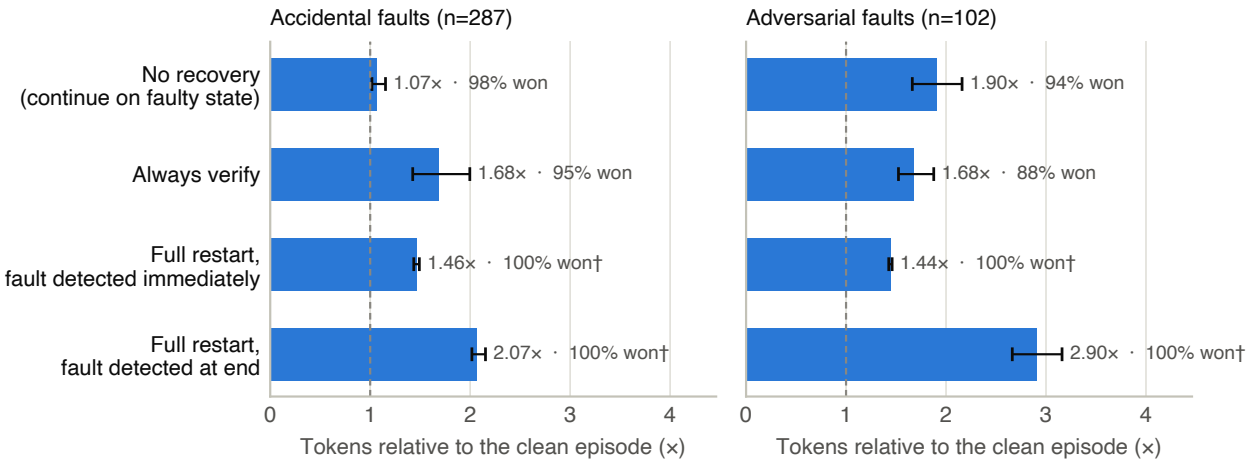


Figure 3: Tokens needed to finish an episode under each response to a fault, relative to the clean episode (mean, 95% CI over games), with the share of tasks won, on the identical seed-42 mid-position schedule. †Restart success is the executed live-restart rate (18/18).

Figure 3 compares four responses to the same faults.

- **Full restart** always recovers the task but repeats all pre-fault work: 14%, 46% and 75% of the clean episode's tokens for early, mid and late faults. It costs 1.4–1.5× a clean episode if the fault is detected immediately and 2.1–2.9× if it is detected at the end.
- **Always-verify** blocked every unauthorized and irreversible action: 154 and 8 under the baseline, 0 and 0 with the verifier. It costs 1.33× even on clean episodes.
- **But always-verify lost more tasks**: 27 of 389 against the baseline's 11, with only 2 lost by both. 23 of the 27 end in `verification_rejected`: the verifier correctly notices that a plan relies on corrupted state, but the planner keeps planning from that state until the attempt limit stops the episode. Verification detects faults; it does not recover from them.

5 Phase 2 direction

In Phase 2 we will build **selective recovery**: dependency-aware invalidation with an influence-aware early cutoff, plus world-state restoration. As the course specifies, every method is told which result is faulty.

- **Structure overestimates influence.** A fired fault has on average 35.2 structurally downstream calls, of which only 21.6 (61%) change; for accidental faults the share is 37–50%. Influence-aware recovery recomputes in dependency order and stops along a branch once a recomputed output equals the old one. Sorted fact lists make that comparison exact and cheap.
- **Recomputation alone is not enough.** In 47% of fired faults the executed actions diverged. ALFWorld replays deterministically given the action prefix (1,164 of 1,164 matched), so Phase 2 resets the environment, replays the valid prefix while checking observation hashes, and resumes from the first affected step. A non-resettable mode compensates with inverse actions instead. The 24 episodes with irreversible actions are reported as unrecoverable, not as recovered.
- **Comparisons and metrics.** Full restart and always-verify, under the same schedule and budget. We measure recovered task correctness and safety (not exact equality, given live non-determinism), recomputed calls, retained work, recovery latency, tracking overhead at several trace granularities, and an audit that re-executes every reused result to check it is still valid.
- **Fuzzing.** The proposal's fuzzing-guided fault discovery is moved to Phase 2. The systematic schedule above serves as its non-fuzzing baseline.

6 Limitations

- All agents share one model, and constraints and the executor are deterministic by design.
- Fault runs cover all positions only for seed 42, and always-verify only the mid position. CIs resample 17–18 games.
- The unauthorized-action policy is a simulation based on object types the task does not name.
- Latencies are logical (recorded model time plus measured non-model time, excluding rate-limit waits). Only the executed restart measures live latency: 27.0 s wall time vs. 25.7 s of model time.
- Output equality is exact, so semantically equal but differently worded outputs count as changed. This overstates influence, making the 61% figure conservative for Phase 2.
- The injected faults are synthetic.
- Two infrastructure failures were kept and rerun: one transient budget rejection from the API proxy, and four runs hit by a cache-write race in our runner, since fixed.

7 Reproducibility and cost

Everything ran on a laptop (CPU only). The binding constraint is the API rate limit (100 requests/min), not compute. All Phase 1 work used 48.6k API calls (46.3M tokens), $10.65 of TritonAI credit; the final evaluation ran for about 11 hours. `experiments/phase1.sh` reproduces all results and is resumable; `analysis/report.py` and `analysis/figures.py` regenerate the tables and figures. The repository README documents the trace format and the fault and runner command-line options.